

Linked List

**WHEN YOU FAIL TO
BUILD YOUR LINKED LIST**



Linear Data Structures

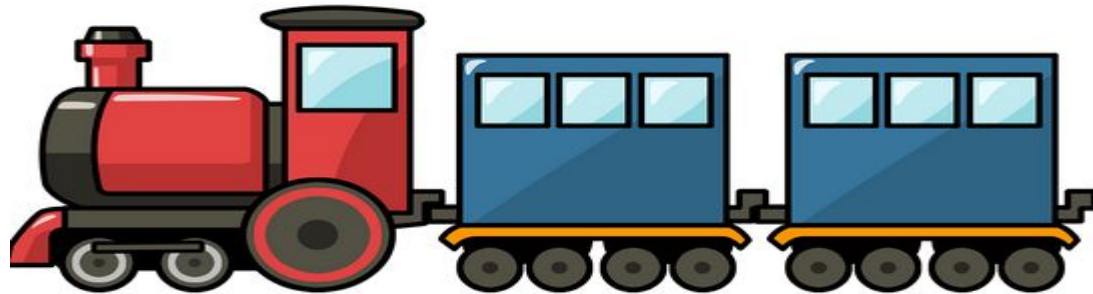
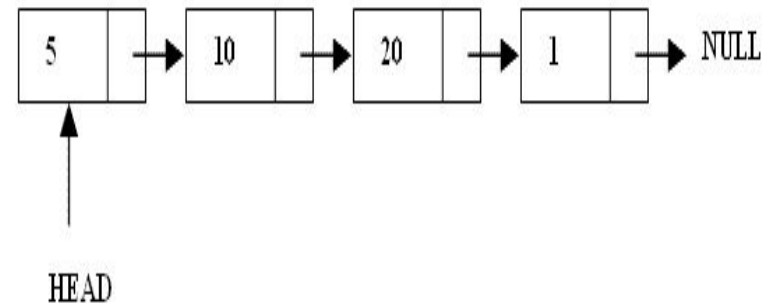
Definition

A data structure is said to be linear if its elements form a sequence or a linear list.

- Examples:
 - Array
 - Linked List
 - Stacks
 - Queues
- **Operations on linear Data Structures**
 - Traversal : Visit every part of the data structure
 - Search : Traversal through the data structure for a given element
 - Insertion : Adding new elements to the data structure
 - Deletion : Removing an element from the data structure.
 - Sorting : Rearranging the elements in some type of order(e.g Increasing or Decreasing)
 - Merging : Combining two similar data structures into one

What are Linked Lists

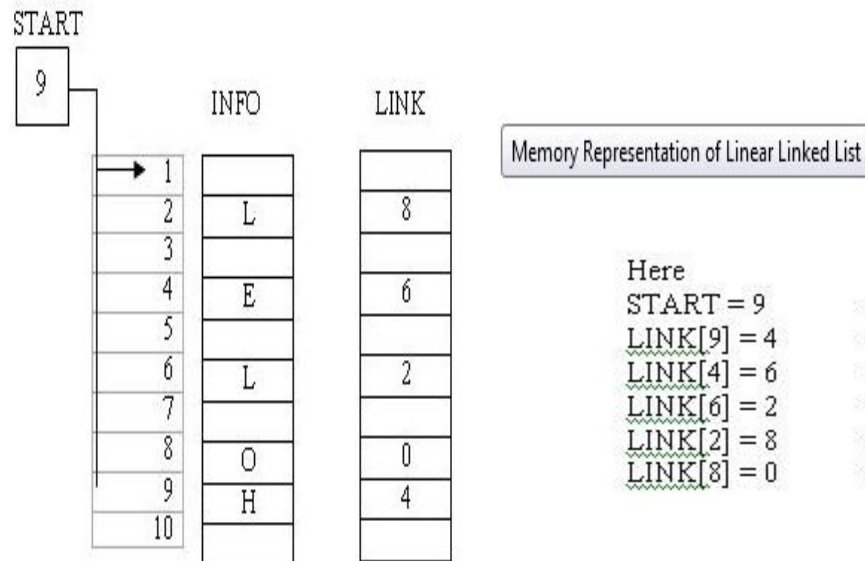
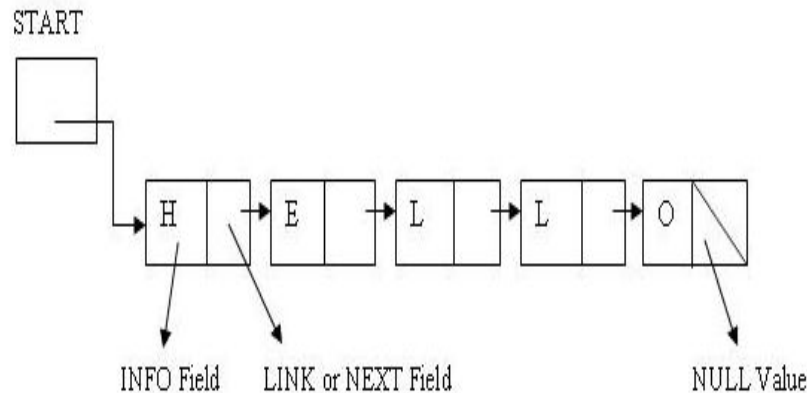
- A linked list is a linear data structure.
- A chain of structures make up linked lists called Node.
- Nodes are made up of data and a pointer to another node.
- Usually the pointer points to a structure of the same type as itself



Arrays Vs Linked Lists

Arrays	Linked list
Fixed size: Resizing is expensive	Dynamic size
Insertions and Deletions are inefficient: Elements are usually shifted	Insertions and Deletions are efficient: No shifting
Random access i.e., efficient indexing	No random access ☐ Not suitable for operations requiring accessing elements by index such as sorting
No memory waste if the array is full or almost full; otherwise may result in much memory waste.	Since memory is allocated dynamically(acc. to our need) there is no waste of memory.
Sequential access is faster [Reason: Elements in contiguous memory locations]	Sequential access is slow [Reason: Elements not in contiguous memory locations]

Representation of linked list in memory



Here

START = 9

LINK[9] = 4

LINK[4] = 6

LINK[6] = 2

LINK[2] = 8

LINK[8] = 0

=>

=>

=>

=>

=>

=>

INFO[9] = H is the first character.

INFO[4] = E is the second character.

INFO[6] = L is the third character.

INFO[2] = L is the fourth character.

INFO[8] = O is the fifth character.

The NULL value, so the LIST ends here.

Types of lists

There are two basic types of linked list

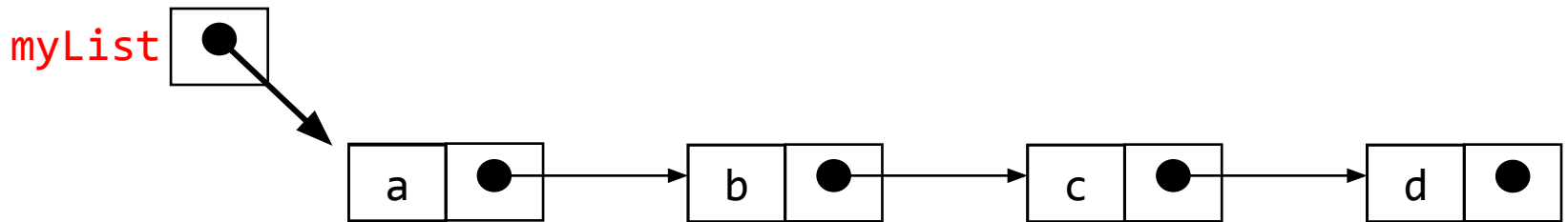
- Singly Linked list
- Doubly linked list
- Circular linked list

Singly Linked List

- Each node has only one link part
- Each link part contains the address of the next node in the list
- Link part of the last node contains NULL value which signifies the end of the node

Schematic representation

- Here is a singly-linked list (SLL):



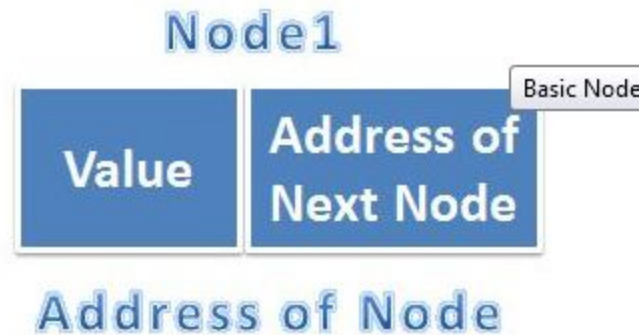
- Each node contains a value(data) and a pointer to the next node in the list
- `myList` is the header pointer which points at the first node in the list

Basic Operations on a list

- Creating a List
- Inserting an element in a list
- Deleting an element from a list
- Searching a list
- Reversing a list

Creating a node

```
struct node {  
    int val; // A simple node of a linked list  
    struct node * next; // pointer to the next address  
};
```

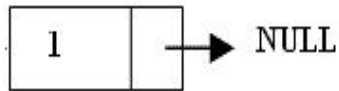


```

void create()
{
    item * curr, * head,*tail;
    head = NULL;
    // beginning of loop
    curr = (item *)malloc(sizeof(item));
    curr->val =read value;
    curr->next = NULL;

    if (head == NULL)
    {
        head = curr;
        tail = curr;
    }
    else {
        tail->next = curr;
        tail = curr;
    }
    // end of loop;
}

```



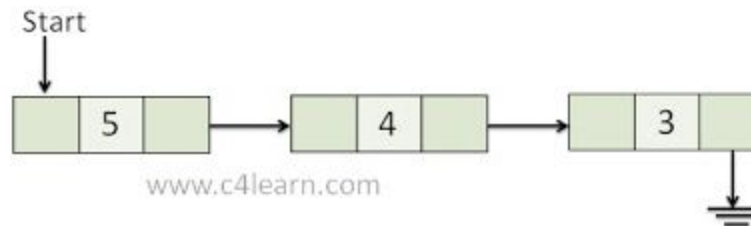
To display

```
void display()
{
    curr = head;
    printf( "\n");
    while(curr != NULL) {
        printf(" %d---> ", curr->val);
        curr = curr->next ;
    }
}
```

Traversing a Linked List

Traversing linked list means visiting each and every node of the Singly linked list. Following steps are involved while traversing the singly linked list –

- Firstly move to the first node
- Fetch the data from the node and perform the operations such as arithmetic operation or any operation depending on data type.
- After performing operation, advance pointer to next node and perform all above steps on Visited node.



Inserting the node in a SLL

There are 3 cases here:-

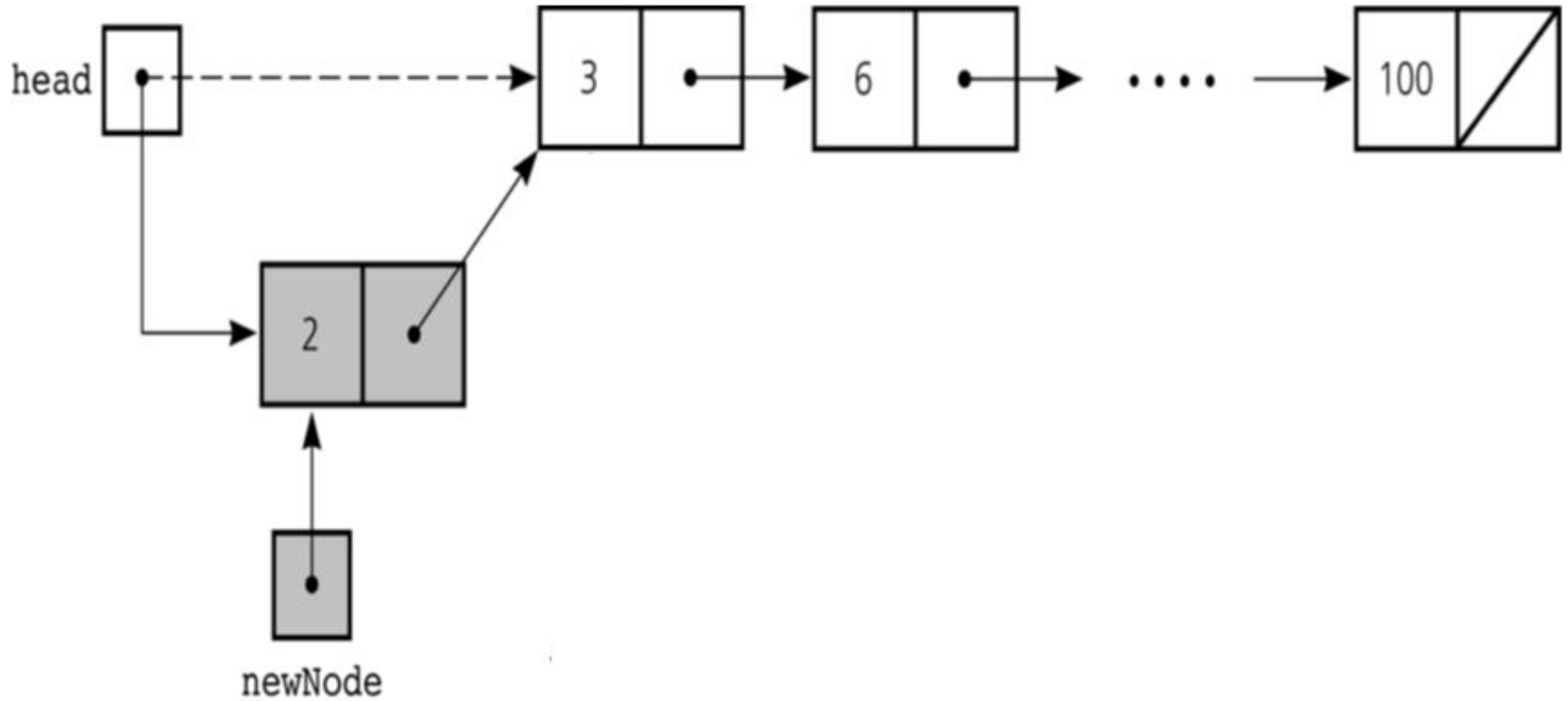
- Insertion at the beginning
- Insertion at the end
- Insertion after a particular node

Insertion at the beginning

There are two steps to be followed:-

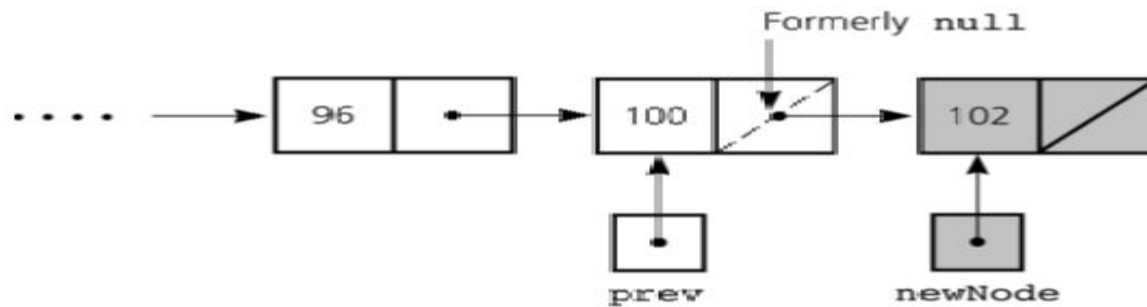
- a) Make the next pointer of the new node point towards the first node of the list
- b) Make the start pointer point towards this new node
 - If the list is empty simply make the start pointer point towards the new node;

Insertion at the beginning



Inserting at the end

Here we simply need to make the next pointer of the last node point to the new node

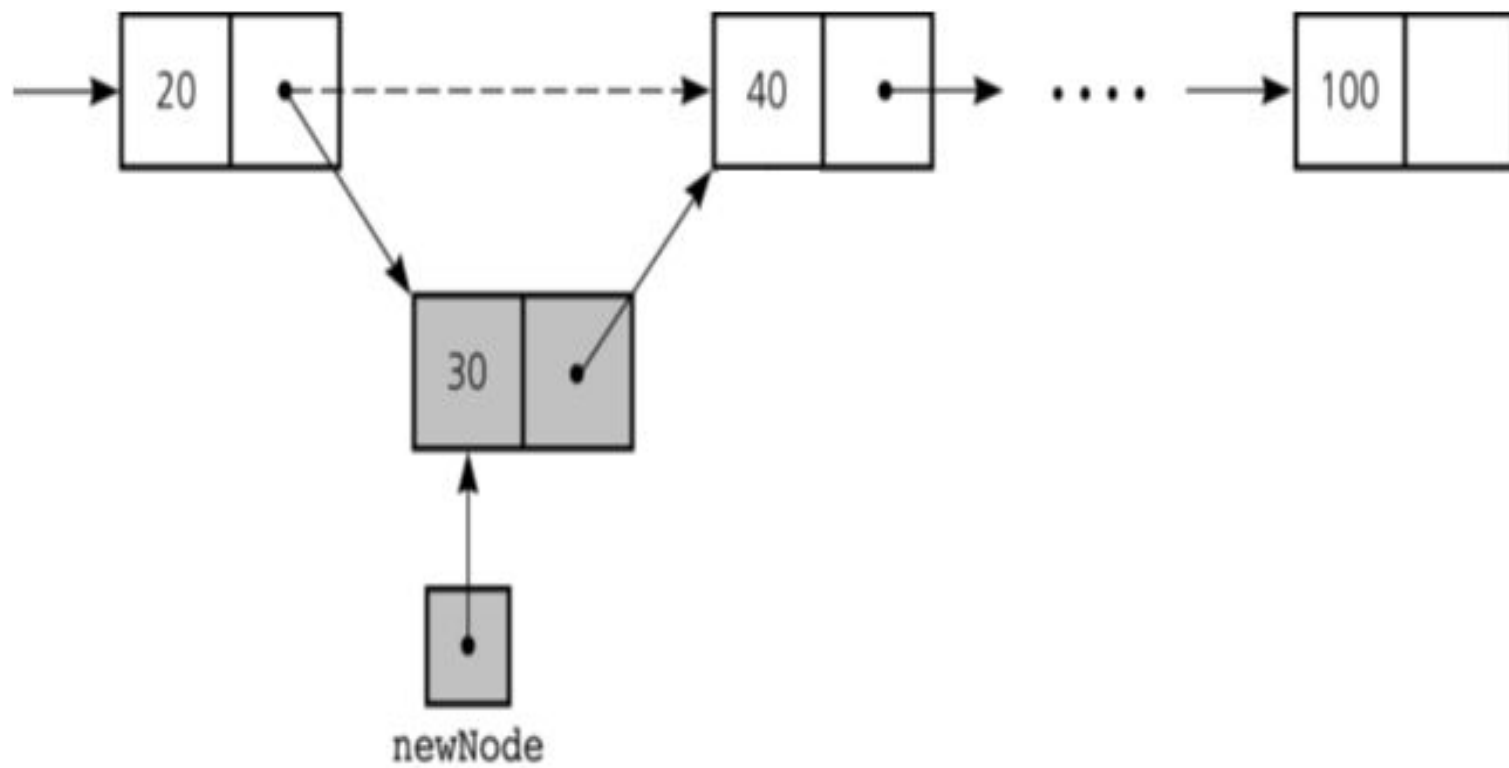


```
void insert_at_end() {  
    struct node *newnode,*current,*temp;  
    newnode=(struct node *)malloc(sizeof(struct node));  
    scanf("%d",&new_node->data);  
    newnode->next=NULL;  
    if(start==NULL) {  
        start=newnode;  
        current=newnode; }  
    else {  
        temp = start;  
        while(temp->next!=NULL) {  
            temp = temp->next;  
        }  
        temp->next = newnode; } }
```

Inserting after an element

Here we again need to do 2 steps :-

- Make the next pointer of the node to be inserted point to the next node of the node after which you want to insert the node
- Make the next pointer of the node after which the node is to be inserted, point to the node to be inserted



```
void insert_mid() {  
    int pos,i; struct node *new_node,*current,*temp,*temp1;  
    new_node=(struct node *)malloc(sizeof(struct node));  
    printf("\nEnter the data : "); scanf("%d",&new_node->data);  
    new_node->next=NULL;  
    printf("\nEnter the position : "); scanf("%d",&pos);  
  
    if(start==NULL) { start=new_node; current=new_node; }  
    else { temp = start;  
        for(i=1;i< pos-1;i++) {  
            temp1 = temp;  
            temp=temp->next; }  
            temp1->next = newnode;  
            newnode->next=temp;  
        }  
    }
```

Deleting a node in SLL

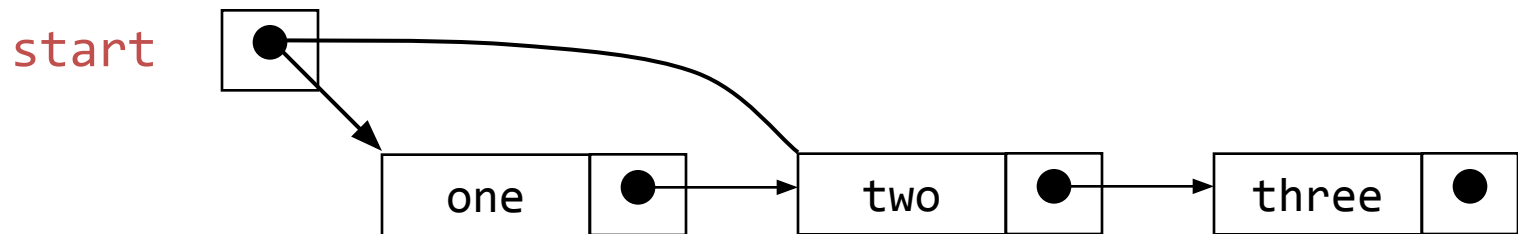
Here also we have three cases:-

- Deleting the first node
- Deleting the last node
- Deleting the intermediate node

Deleting the first node

Here we apply 2 steps:-

- Making the start pointer point towards the 2nd node
- Deleting the first node using **delete** keyword

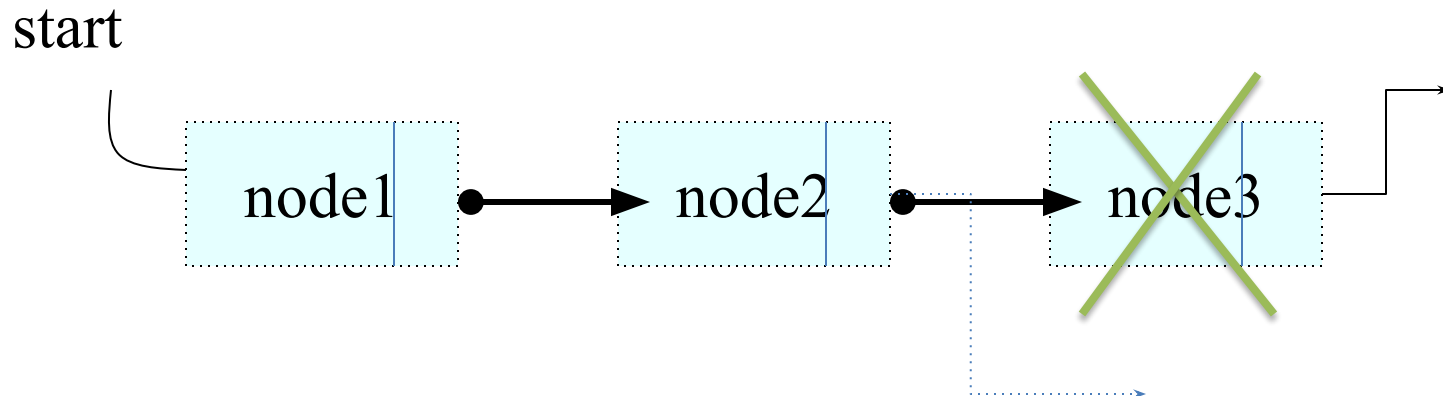



```
void del_beg() {  
    if (start == NULL) { // Check if the list is empty  
        cout << "List is empty, nothing to delete!" << endl;  
        return;  
    }  
  
    node* temp = start; // Store current head  
    start = start->next; // Move head to the next node  
    delete temp; // Free memory  
}
```

Deleting the last node

Here we apply 2 steps:-

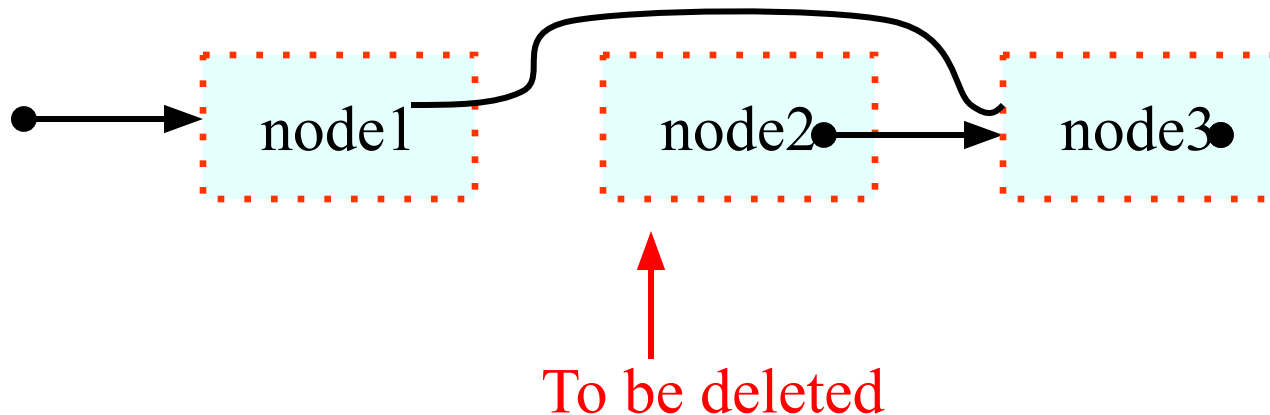
- Making the second last node's next pointer point to NULL
- Deleting the last node via delete keyword



```
void delete_at_end() {  
    if (start == NULL) { // Check if the list is empty  
        std::cout << "List is empty, nothing to delete!" << std::endl;  
        return;  
    }  
    struct node *temp = start, *temp1 = NULL;  
    // Case 1: If the list has only one node  
    if (start->next == NULL) {  
        delete start;  
        start = NULL; // Update start to NULL  
        return;  
    }  
    // Case 2: Traverse to find the second-last node  
    while (temp->next != NULL) {  
        temp1 = temp;  
        temp = temp->next;  
    }  
    temp1->next = NULL; // Update second-last node's next to NULL  
    delete temp;      // Delete last node  
}
```

Deleting a particular node

Here we make the next pointer of the node previous to the node being deleted ,point to the successor node of the node to be deleted and then delete the node using **delete** keyword



```
void delete_any(int key) {
    struct node *temp, *temp1;

    if (start == NULL) return; // Check if list is empty
    temp = start;

    // Case 1: If the node to be deleted is the first node
    if (temp->data == key) {
        start = temp->next; // Move start to next node
        delete temp; // Free memory
        return;
    }

    // Traverse the list to find the node to be deleted
    while (temp != NULL && temp->data != key) {
        temp1 = temp;
        temp = temp->next;
    }

    // If key is not found in the list
    if (temp == NULL) return;

    // Delete the node
    temp1->next = temp->next;
    delete temp;
}
```

Searching a SLL

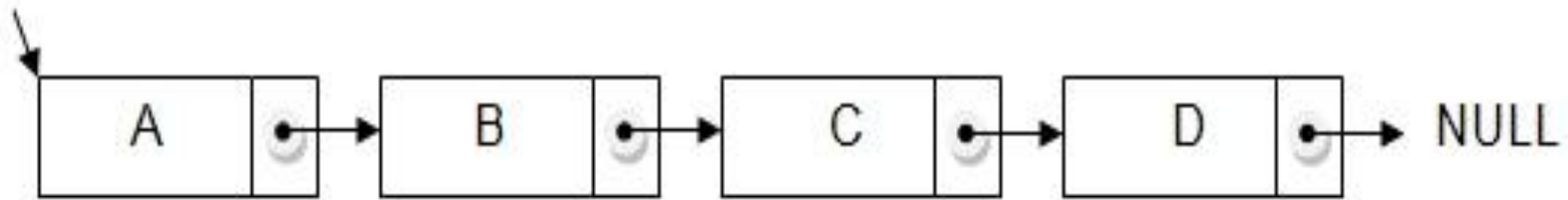
- Searching involves finding the required element in the list
- We can use various techniques of searching like linear search or binary search where binary search is more efficient in case of Arrays
- But in case of linked list since random access is not available it would become complex to do binary search in it
- We can perform simple linear search traversal

In linear search each node is traversed till the data in the node matches with the required value

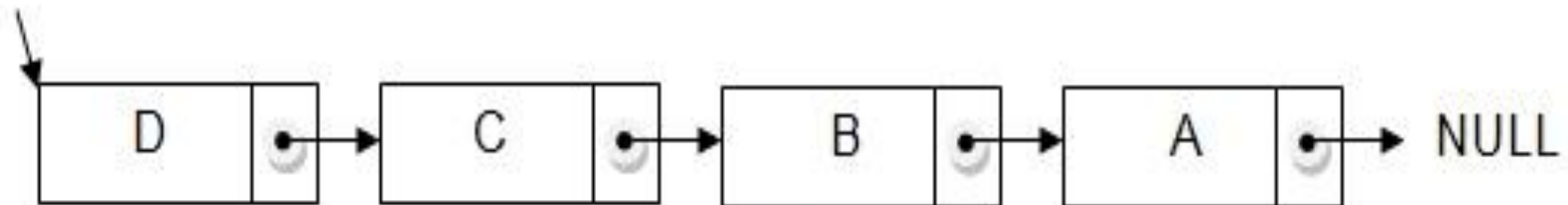
```
void search(int x)
{
    node*temp=start;
    while(temp!=NULL)
    {
        if(temp->data==x)
        {
            cout<<"FOUND "<<temp->data;
            break;
        }
        temp=temp->next;
    }
}
```

Reversing a linked list

- We can reverse a linked list by reversing the direction of the links between 2 nodes



Input



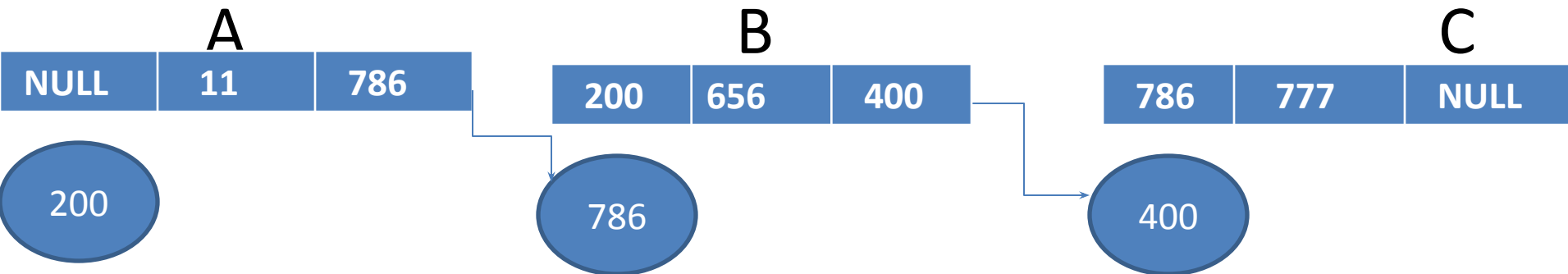
Output

Doubly Linked List

1. **Doubly linked list** is a linked data structure that consists of a set of sequentially linked records called nodes.
2. Each node contains three fields ::
 - : one is data part which contain data only.
 - :two other field is links part that are point or references to the previous or to the next node in the sequence of nodes.
3. The beginning and ending nodes' **previous** and **next** links, respectively, point to some kind of terminator, typically a sentinel node or null to facilitate traversal of the list.

NODE

previous	data	next
----------	------	------



A doubly linked list contains three fields: an integer value, the link to the next node, and the link to the previous node.

DLL's compared to SLL's

● Advantages:

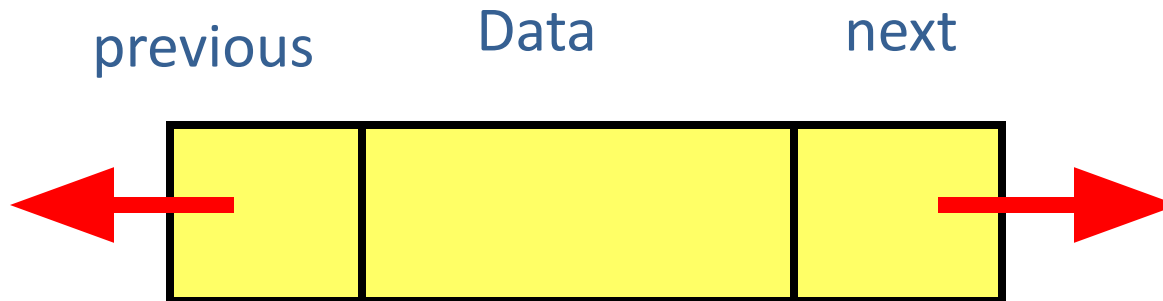
- We can traverse in both directions i.e. from starting to end and as well as from end to starting.
- It is easy to reverse the linked list.
- If we are at a node, then we can go to any node. But in linear linked list, it is not possible to reach the previous node.

● Disadvantages:

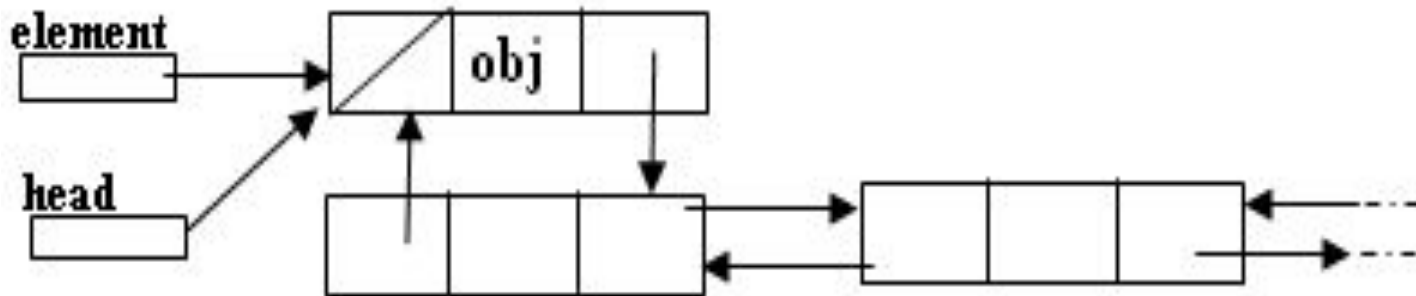
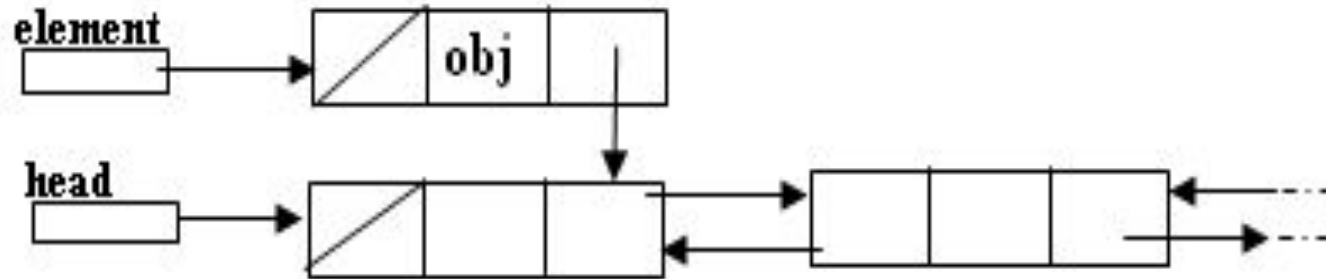
- Requires more space
- List manipulations are slower (because more links must be changed)
- Greater chance of having bugs (because more links must be manipulated)

Structure of DLL

```
struct node
{
    int data;
    node*next;
    node*previous; //holds the address of previous node
};
```



Inserting at beginning



Inserting at beginning

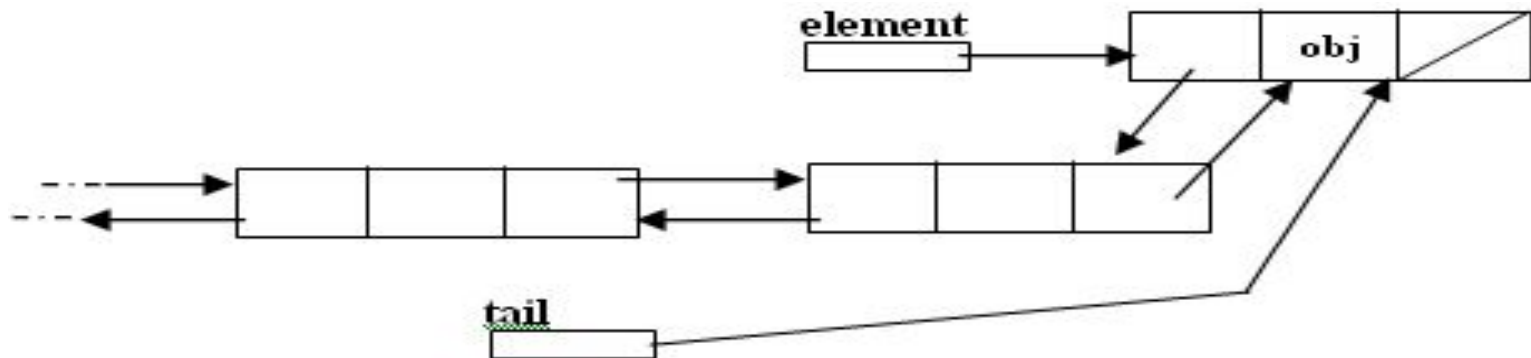
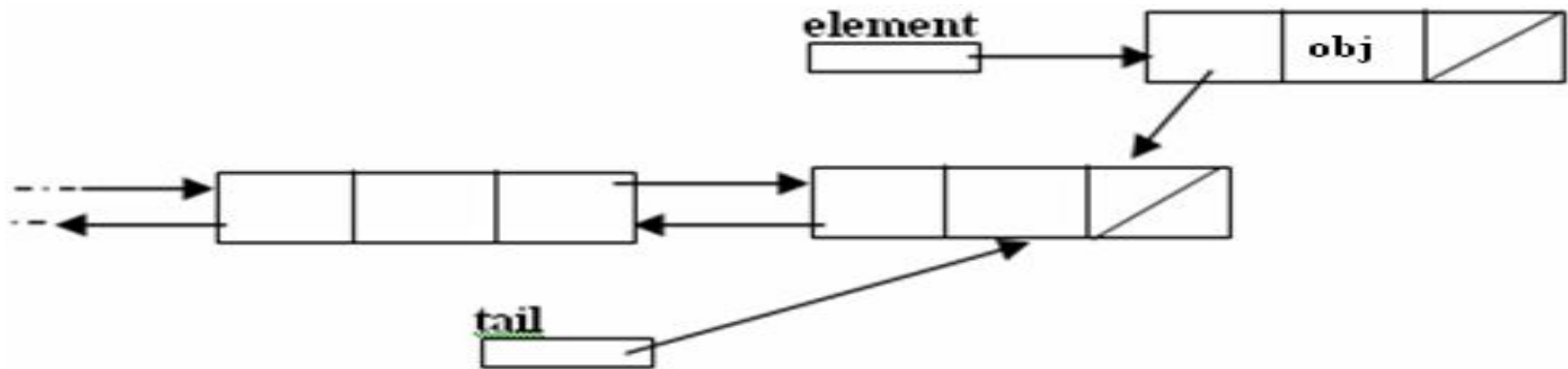
```
void insertAtBeginning(struct Node** head, int newData) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = newData;
    newNode->prev = NULL;           // Since we are inserting at the beginning

    if (*head == NULL) {          // Check if the DLL is empty
        newNode->next = NULL;
    } else {
        newNode->next = *head;    // Point new node's next to the current head
        (*head)->prev = newNode; // Update current head's prev to point to
        new node
    }

    // Finally, update the head pointer to the new node
    *head = newNode;
}
```

Inserting at the end



Inserting at the end

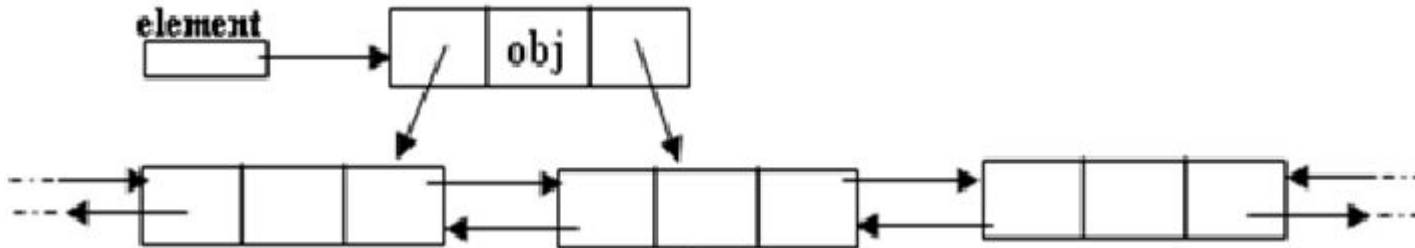
```
void insertAtEnd(struct Node** head, int newData) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = newData;
    newNode->next = NULL;    // Since this will be the last node, its next should be NULL

    if (*head == NULL) {
        // Case 1: If the DLL is empty, the new node becomes the only node in the list
        newNode->prev = NULL;
        *head = newNode;
        return;
    }

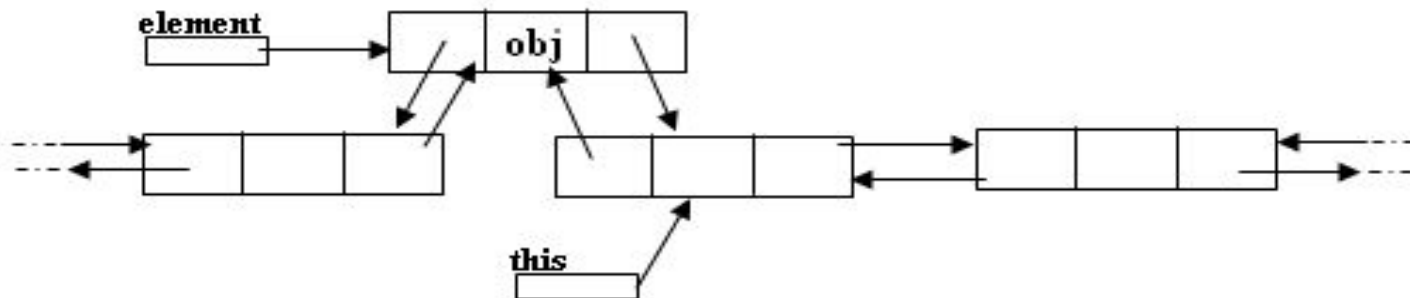
    struct Node* temp = *head;
    while (temp->next != NULL) {        // Case 2: If DLL is not empty, traverse to the last node
        temp = temp->next;
    }

    temp->next = newNode; // Link last node's next to the new node
    newNode->prev = temp; // Link new node's prev to the last node
}
```


Inserting after a node



Making next and previous pointer of the node to be inserted point accordingly



Adjusting the next and previous pointers of the nodes b/w which the new node accordingly

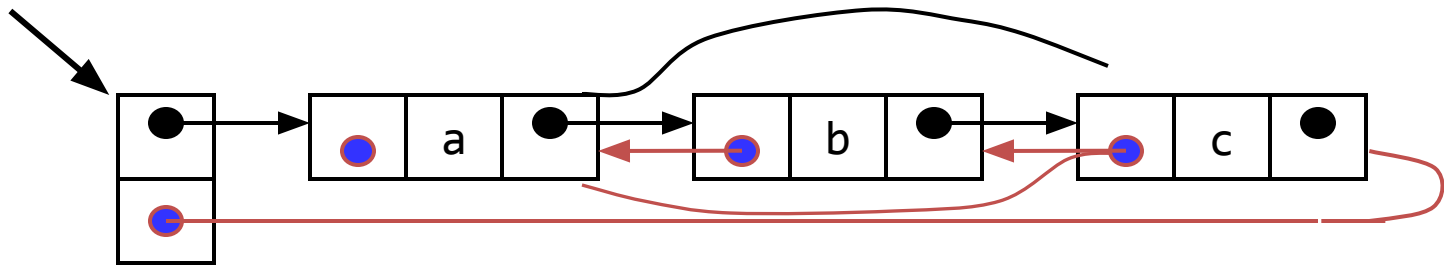
Inserting after a node

```
void insertAfterPosition(struct Node** head, int position, int newData) {  
  
    struct Node* temp = *head;  
    int count = 0;  
  
    while (temp != NULL && count < position) {  
        temp = temp->next;  
        count++;  
    }  
  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = newData;  
  
    // Adjust pointers to insert the new node  
    newNode->next = temp->next;  
    newNode->prev = temp;  
    temp->next = newNode;  
  
    // If the new node is not the last node, update its next node's prev pointer  
    if (newNode->next != NULL) {  
        newNode->next->prev = newNode;  
    }  
}
```

Deleting a node

- Node deletion from a DLL involves changing *two* links
- In this example, we will delete node b

myDLL



- We don't have to do anything about the links in node b
- Garbage collection will take care of deleted nodes
- Deletion of the first node or the last node is a special case

Deleting a node

```
void deleteNodeAtPosition(struct Node** head, int position) {

    struct Node* temp = *head;
    int count = 0;
    while (temp != NULL && count < position) {
        temp = temp->next;
        count++;
    }

    // Case 1: If the node to be deleted is the head node
    if (*head == temp) {
        *head = temp->next; // Move head to the next node
    }

    // Case 2: If temp is not the last node, update its next node's prev pointer
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }

    // Case 3: If temp is not the first node, update its previous node's next pointer
    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }

    free(temp);
}
```

APPLICATIONS OF LINKED LIST

- Applications that have an MRU(most recently used) list (a linked list of file names)
- The cache in your browser that allows you to hit the BACK button (a linked list of URLs)
- Undo functionality in Photoshop or Word (a linked list of state)
- A linked list can implement a stack, hash table, and binary tree.

Review Questions

- ❖ Suppose you have a list of integer values. You need to separate the values into two lists: one containing odd numbers and the other containing even numbers. However, the relative order of elements must remain unchanged in both lists.
 - Which data structure would be best suited for this task and why?
 - How would you efficiently implement this using a linked list?
- ❖ Consider the following code snippet:

```
void exammarks()
{
    DLL *s1 = new DLL();
    s1->insert(1, 3.2);
    s1->insert(2, 4.0);
    s1->insert(3, 3.87);
}
```

If DLL represents a doubly linked list, how should the insert() function be implemented to allow insertion at a given position? Write a delete() function that removes values less than a certain given value from the list.

Review Questions

- ❖ You are designing a music player where users can navigate forward and backward between songs.
 - Which linked list type is most suitable and why?
 - How would you implement the 'Next' and 'Previous' song functionality with your choice of linked list?
 - What if the playlist loops back to the first song after reaching the last song?

- ❖ Implement a function to insert an element at the head of a singly linked list.

```
struct Node {  
    int data;  
    Node* next;  
};
```

Explain the time complexity of your implementation. Do the same for other operations like deleting, searching, etc.

Review Questions

- ❖ You are developing a navigation system where paths are represented as a singly linked list, where each node points to the next location. You suspect that the system might have a loop (i.e., the path could circle back on itself).

How would you detect if there is a loop in this singly linked list representing the path? What would be the consequences if you don't detect the loop?

- ❖ If a doubly linked list uses extra memory to store back pointers, is it worth the trade-off? Under what type of workload is it beneficial? Give examples.